

1) Fraud Transaction detection (linear search)

	1201	1305	1450	1408	1502
i =	0	1	2	3	4

find fraudId

```

int fraudId; cin >> fraudId;
for (int i=0; i<N; i++) {
    if (arr[i] == fraudId) {
        cout << "Fraud Transaction found" << endl;
    } else {
        cout << "Transaction not found" << endl;
    }
}
return;
    
```

2) Server Log Search (Binary Search)

	1001	1005	1010	1015	1020	1030	1040
i =	0	1	2	3	4	5	6
	low			mid			high

```

while (low <= high) {
    mid = low + (high - low) / 2;
    }
    
```

```

if (arr[mid] == target) {
    cout << "log found" << endl;
}
    
```

```

else if (arr[mid] < target) {
    low = mid + 1;
}
    
```

```

else { high = mid - 1; }
}
    
```

```

cout << "log not found" << endl;
    
```


Date / /

3) Sort Employee Salaries (Bubble Sort)
 Pass 1
 $j=1$
 45000 32000 55000 28000 60000
 $arr[j]$ $arr[j+1]$
 swap
 $arr[j] > arr[j+1]$

$j=2$
 32000 45000 55000 28000 60000
 No swap

$j=3$
 32000 45000 55000 28000 60000
 swap

$j=4$
 32000 45000 28000 55000 60000
 fixed largest element

Pass 2 →

$j=1$
 32000 45000 28000 55000 60000
 No swap

$j=2$
 32000 45000 28000 55000 60000
 swap

$j=3$
 32000 28000 45000 55000 60000
 do not swap

$j=4$
 32000 28000 45000 55000 60000

Pass 3 →

$j=1$
 32000 28000 45000 55000 60000
 swap

$j=2$
 28000 32000 45000 55000 60000
 do not swap

$j=3$
 28000 32000 45000 55000 60000

Pass 4 →

$j=1$
 28000 32000 45000 55000 60000
 do not swap

$j=2$
 28000 32000 45000 55000 60000


```

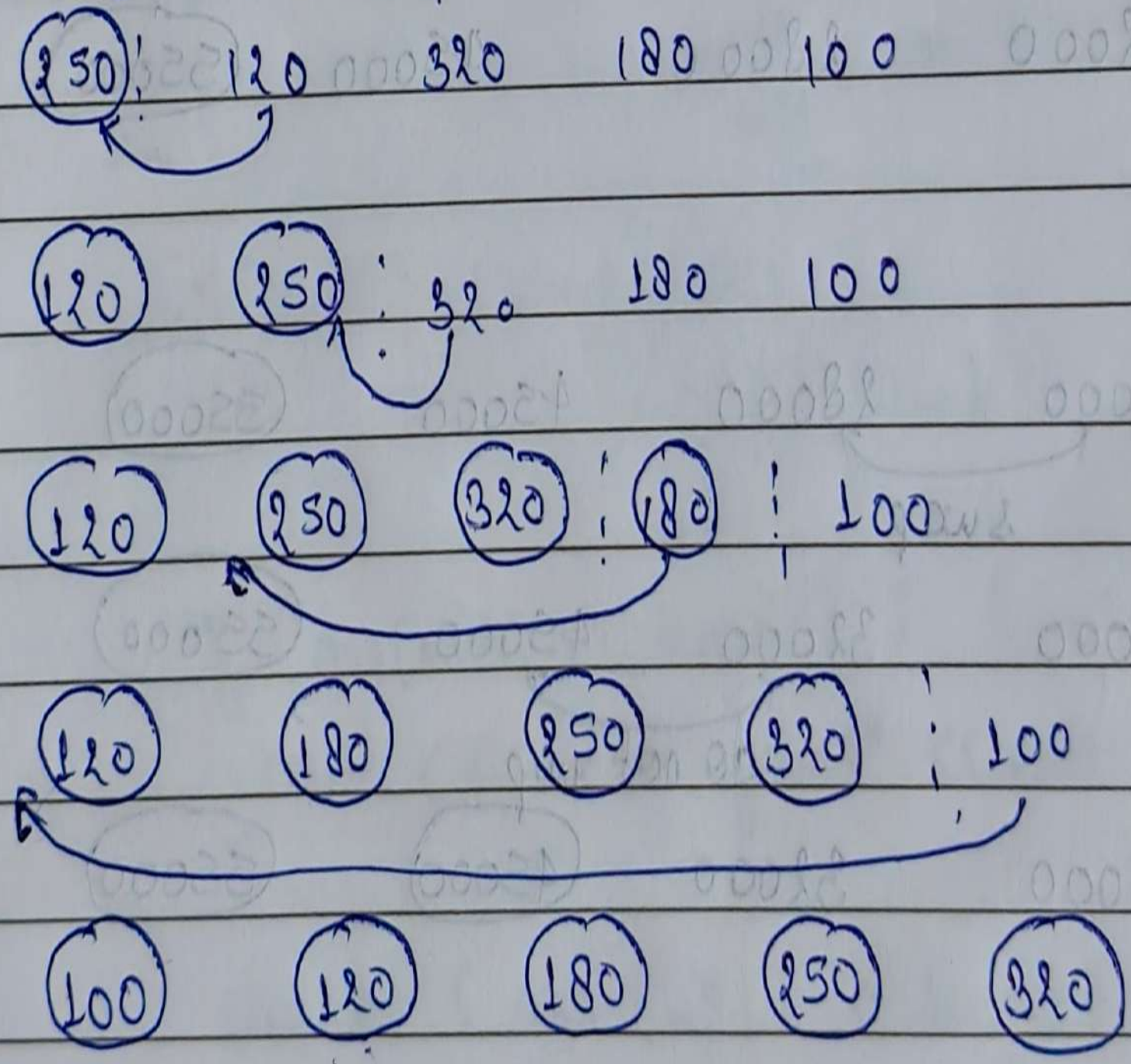
for (i = 0; i < N; i++)
{
    for (j = 0; j < N - i - 1; j++)
    {
        if (arr[j] > arr[j+1])
        {
            int temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
        }
    }
}

```

4) Sort website response time (Insertion time)

250 120 320 180 100

compare j & j+1 - check for elements & insert it at correct place



```

for (int i = 1; i < N; i++)
{
    int key = arr[i];

```



```
int j = i - 1;
```

```
while (j >= 0 & arr[j] > key)
```

```
{
```

```
    arr[j+1] = arr[j]
```

```
    j--;
```

```
}
```

```
arr[j+1] = key;
```

```
}
```

```
cout << arr;
```

5) Bug Reports (Selection Sort)

9 2 8 1 5

① 9 2 8 5

① ② 9 8 5

① ② ⑤ 9 8

① ② ⑤ ⑧ ⑨

```
for (i = 0; i < N - 1; i++)
```

```
{
```

```
    min = i;
```

```
    for (j = i + 1; j < N; j++)
```

```
    {
```

```
        if (arr[j] < arr[min])
```

```
        {
```

```
            min = j;
```

```
        }
```


}

temp = arr[i];

arr[i] = arr[min];

arr[min] = temp;

}

(Bubble Sort)

Time Complexity

2 1 3 4 5

2 3 4 5 1

2 3 4 1 5

3 4 1 2 5

3 4 1 2


```
EmployeeSalary.cpp      EmployeeRate.cpp      fraudTransac

#include <iostream>
using namespace std;

bool findFraudId(int arr[], int N, int fraudId){
    for(int i=0; i<N; i++){
        if(arr[i]==fraudId)
            return true;
    }
    return false;
}

int main(){
    int N, fraudId;
    cin>>N;
    int arr[100];

    for(int i=0; i<N; i++){
        cin>>arr[i];
    }
    cin>>fraudId;

    findFraudId(arr,N,fraudId);

    if(findFraudId(arr,N,fraudId)){
        cout<<"Fraud Transaction found"<<endl;
    }else{
        cout<<"Transaction not found"<<endl;
    }
    return 0;
}
```



```
if(findFraudId(arr, N, fraudId))
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

power

```
PS C:\PD\DSA\Searching> g++ fraudTransactionDetection.cpp
```

```
PS C:\PD\DSA\Searching> ./a.exe
```

5

1201

1305

1450

1408

1502

1450

Fraud Transaction found

```
PS C:\PD\DSA\Searching>
```


X ddb:preagbort

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

powershell

```
PS C:\PD\DSA\Searching> ./a.exe
```

7

1001

1005

1010

1015

1020

1030

1040

1015

Log Found

```
PS C:\PD\DSA\Searching> █
```



Search




```
#include <iostream>
using namespace std;
```

```
void bubbleSort(int arr[], int N){
    for(int i=0; i<N; i++){
        for(int j=0; j<N-i-1; j++){
            if(arr[j] > arr[j+1]) {
                int temp=arr[j];
                arr[j]=arr[j+1];
                arr[j+1]=temp;
            }
        }
    }
    for(int i=0; i<N; i++){
        cout<<arr[i]<<" ";
    }
    return;
}
```

```
int main() {
    int N;
    cin>>N;
    int arr[1000];
    for(int i=0; i<N; i++) {
        cin>>arr[i];
    }
    bubbleSort(arr,N);
    return 0;
}
```

I



PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\PD\DSA\Searching> g++ employeeSalaries.cpp

PS C:\PD\DSA\Searching> ./a.exe

5

45000

32000

55000

28000

60000

28000 32000 45000 55000 60000

PS C:\PD\DSA\Searching> █

I


```

#include <iostream>
using namespace std;
void insertionSort(int arr[], int N){
    for(int i=1; i<N; i++){
        int key=arr[i];
        int j=i-1;
        while (j>=0 && arr[j]>key){
            arr[j+1]=arr[j];
            j--;
        }
        arr[j+1]=key;
    }
    for(int i=0; i<N; i++){
        cout<<arr[i]<<" ";
    }
    return;
}
int main () {
    int N;
    cin>>N;
    int arr[1000];
    for(int i=0; i<N; i++) {
        cin>>arr[i];
    }
    insertionSort(arr,N);
    return 0;
}

```

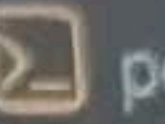

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS



PS C:\PD\DSA\Searching> g++ websiteResponse.cpp

PS C:\PD\DSA\Searching> ./a.exe

5

250

120

320

180

100

100 120 180 250 320

PS C:\PD\DSA\Searching> █


```
0  #include <iostream>
1  using namespace std;
2  void selectionSort(int arr[], int N){
3      int i,j, mini, temp;
4      for(i=0; i<N-1; i++){
5          mini=i;
6          for(j=i+1; j<N; j++){
7              if (arr[j]< arr[mini]){
8                  mini=j;
9              }
10         }
11         temp=arr[i];
12         arr[i]=arr[mini];
13         arr[mini]=temp;
14     }
15     for(int i=0; i<N; i++){
16         cout<<arr[i]<<" ";
17     }
18     return;
19 }
20 int main () {
21     int N;
22     cin>>N;
23     int arr[1000];
24     for(int i=0; i<N; i++) {
25         cin>>arr[i];
26     }
27     selectionSort(arr,N);
28     return 0;
29 }
```



PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS



```
PS C:\PD\DSA\Searching> g++ bugReports.cpp
```

```
PS C:\PD\DSA\Searching> ./a.exe
```

5

9

2

8

1

5

1 2 5 8 9

```
PS C:\PD\DSA\Searching> █
```



Search

